2026-10-18 11:36:12 You

Create a detailed explanation but only point to point no extra things i want simple and clear understanding and basic practical examples for each

🟡 Level 3 — Variables & Data

Very important for real projects.

26. **Variable definition**
27. **Variable precedence**
28. **Host variables**
29. **Group variables**
30. **Command-line variables (`-e`)**
31. **Registered variables**
32. **Facts**
33. **Magic variables**
34. **Jinja2 templating**
35. **Conditions (`when`)**
36. **Loops**
37. **Loop variables**
38. **Filters**

ChatGPT gpt-6

Ansible Level 3 — Variables & Data (26–38)

Goal: Understand each concept, know when to use it, and practice with a simple example.

26. Variable Definition

Meaning: A variable stores a value that you can reuse instead of hardcoding it everywhere.

Example: Instead of writing `nginx` repeatedly, store it in a variable.

File: `playbook.yml`

```yaml
- name: Variable example
  hosts: web
  vars:
    package_name: nginx

  tasks:
    - name: Install package
      ansible.builtin.apt:
        name: "{{ package_name }}"
        state: present
        update_cache: yes
      become: yes
```

Understand:

- `vars:` defines variables inside a play.
- `package_name` is the variable name.
- `nginx` is its value.
- `{{ package_name }}` retrieves the value.
- Ansible uses Jinja2 syntax `{{ }}` to reference variables in most task arguments.

Practical use: Change the package name in one place instead of editing multiple tasks.

---

27. Variable Precedence

Meaning: When the same variable is defined in multiple places, Ansible decides which value to use based on its precedence rules.

Example: Suppose you define `app_port` in two places.

```yaml
# playbook.yml
- name: Precedence example
  hosts: web
  vars:
    app_port: 8080

  tasks:
    - name: Display port
      ansible.builtin.debug:
        msg: "Application port is {{ app_port }}"
```

Run:

```
</> Bash
ansible-playbook -i inventory.ini playbook.yml -e "app_port=9090"
```

Output:

```
</> Plain text
Application port is 9090
```

Why?

- Playbook variable: `8080`
- Command-line extra variable (`-e`): `9090`
- Extra variables normally have higher precedence than play variables.

Common precedence order, simplified from lower to higher:

1. Role defaults (`defaults/main.yml`)
2. Inventory group variables
3. Inventory host variables
4. Playbook variables
5. Task variables
6. Command-line extra variables (`-e`)

This is a simplified list, not the complete precedence hierarchy. Other sources, such as facts, registered variables, role variables, and block variables, also have specific precedence rules.

Practical use: Override a default port, environment name, or version without editing the playbook.

---

28. Host Variables

Where it is declared?

Meaning: Variables assigned to a particular server.

Inventory file?

Example: You have two web servers running different application ports.

File: `inventory.ini`

```
</> INI
```

```
[web]
web01 ansible_host=192.168.1.10 app_port=8080
web02 ansible_host=192.168.1.11 app_port=9090
```

File: `playbook.yml`

```yaml
- name: Host variable example
  hosts: web
  tasks:
    - name: Display server port
      ansible.builtin.debug:
        msg: "{{ inventory_hostname }} uses port {{ app_port }}"
```

Run:

```bash
ansible-playbook -i inventory.ini playbook.yml
```

Example output:

```
web01 uses port 8080
web02 uses port 9090
```

Understand:

- `web01` has its own `app_port`.
- `web02` has its own `app_port`.
- Each host receives its corresponding value.

Practical use: Different IP addresses, ports, application versions, or disk sizes for individual servers.

---

29. Group Variables

Meaning: Variables shared by all servers belonging to a group.

Example: All web servers use the same package and application port.

Folder structure:

```
</> Plain text

ansible-project/
├──── inventory.ini
├──── group_vars/
│     └──── web.yml
└──── playbook.yml
```

File: `inventory.ini`

```
</> INI

[web]
web01
web02
```

File: `group_vars/web.yml`

```
</> YAML

package_name: nginx
app_port: 8080
```

File: `playbook.yml`

```
</> YAML

- name: Group variable example
  hosts: web
  tasks:
    - name: Display shared variables
      ansible.builtin.debug:
        msg: "Install {{ package_name }} on port {{ app_port }}"
```

Understand:

- `group_vars/web.yml` applies to hosts in the `web` group.
- Both servers receive `package_name: nginx`.
- Both servers receive `app_port: 8080`.
- A host-specific variable can override a group variable when the host variable has higher precedence.

Practical use: Share common configurations across development, web, database, or application server groups.

Remember the difference:

Host variables	Group variables
One particular server	Multiple servers in a group
`host_vars/web01.yml`	`group_vars/web.yml`

Where it is Defined?

---

30. Command-Line Variables (`-e`)

Meaning: Pass a variable when running a playbook, without changing the playbook file.

File: **`playbook.yml`**

```yaml
- name: Command-line variable example
  hosts: localhost
  connection: local

  tasks:
    - name: Display environment
      ansible.builtin.debug:
        msg: "Deploying to {{ environment }}"
```

Run:

```bash
ansible-playbook playbook.yml -e "environment=production"
```

Output:

```
Deploying to production
```

You can change it for the next run:

```bash
ansible-playbook playbook.yml -e "environment=staging"
```

Other syntax:

```bash
ansible-playbook playbook.yml -e '{"environment":"production","app_port":8080}'
```

Understand:

- `-e` means extra variables.
- The value is supplied at execution time.
- Extra variables have very high precedence.

Practical use: Deploy the same application to development, staging, or production using one playbook.

---

31. Registered Variables

Meaning: Store the result of a task in a variable so that later tasks can use it.

Example: Execute a command and capture its output.

```yaml
- name: Registered variable example
  hosts: localhost
  connection: local

  tasks:
    - name: Check current user
      ansible.builtin.command: whoami
      register: user_result
      changed_when: false

    - name: Display command output
      ansible.builtin.debug:
        msg: "Current user is {{ user_result.stdout }}"
```

Understand:

- `register: user_result` stores the task result.
- `user_result.stdout` contains the command's standard output.
- `user_result.rc` contains the return code.
- `user_result.stderr` contains standard error.

Practical use: Check whether a service is running, capture a command result, or use a health-check result in a later task.

Important: Registered variables normally contain structured task results, not just a plain string.

---

32. Facts

Meaning: Facts are information Ansible automatically gathers about managed hosts.

Examples include:

- Operating system
- IP addresses
- Memory
- CPU information
- Hostname

Example:

```yaml
- name: Facts example
  hosts: web
  gather_facts: yes

  tasks:
    - name: Display operating system
      ansible.builtin.debug:
        msg: "OS is {{ ansible_facts['distribution'] }}"

    - name: Display hostname
      ansible.builtin.debug:
        msg: "Hostname is {{ ansible_facts['hostname'] }}"
```

Example output:

```
OS is Ubuntu
Hostname is web01
```

Understand:

- `gather_facts: yes` enables fact gathering for the play.

- `ansible_facts['distribution']` provides the operating-system distribution.
- Facts describe the host Ansible is managing.

Practical use: Install `apt` packages on Ubuntu and `dnf` packages on RHEL-based systems.

Example condition:

```yaml
- name: Install nginx on Ubuntu
  ansible.builtin.apt:
    name: nginx
    state: present
  become: yes
  when: ansible_facts['distribution'] == "Ubuntu"
```

Remember: Facts are gathered information; normal variables are values you define yourself.

---

33. Magic Variables

Meaning: Special variables automatically provided by Ansible to describe the inventory, hosts, and current execution context.

You generally do not define these yourself.

Magic variable	Meaning	Example
`inventory_hostname`	Name of the current inventory host	`web01`
`groups`	Hosts organized by inventory group	`groups['web']`
`hostvars`	Variables and available host data indexed by host	`hostvars['web01']['app_port']`
`group_names`	Groups the current host belongs to	`['web', 'production']`
`play_hosts`	Hosts in the current play's active host set	List of host names

Example:

```yaml
```

```yaml
- name: Magic variables example
  hosts: web
  tasks:
    - name: Display current host
      ansible.builtin.debug:
        msg: "Current host is {{ inventory_hostname }}"

    - name: Display web group members
      ansible.builtin.debug:
        msg: "Web servers: {{ groups['web'] }}"
```

Understand:

- `inventory_hostname` identifies the current host in the inventory.
- `groups['web']` gives the host names in the `web` group.
- `hostvars` lets you access another host's variables.

Example of accessing another host:

```yaml
</> YAML

- name: Show another server's port
  ansible.builtin.debug:
    msg: "{{ hostvars['web01']['app_port'] }}"
```

This assumes `web01` exists in the inventory and `app_port` is defined for it.

Practical use: Configure an application using a database server's inventory-defined IP address or other host variables.

---

34. Jinja2 Templating

Meaning: Jinja2 lets you insert variables and expressions into text or configuration files.

Basic syntax:

```jinja2
</> Jinja2

{{ variable_name }}
```

Example 1 — Use a variable in a task:

```yaml
</> YAML
```

```yaml
- name: Jinja example
  hosts: localhost
  connection: local
  vars:
    app_name: payment-api

  tasks:
    - name: Display application name
      ansible.builtin.debug:
        msg: "Deploying {{ app_name }}"
```

Output:

```
Deploying payment-api
```

Example 2 — Generate a configuration file

Folder structure:

```
project/
├──── playbook.yml
└──── templates/
    └──── app.conf.j2
```

File: `templates/app.conf.j2`

```
APP_NAME={{ app_name }}
APP_PORT={{ app_port }}
```

File: `playbook.yml`

```yaml
- name: Template example
  hosts: localhost
  connection: local

  vars:
    app_name: payment-api
    app_port: 8080
```

```
  tasks:
    - name: Generate configuration
      ansible.builtin.template:
        src: templates/app.conf.j2
        dest: /tmp/app.conf
```

Run:

```
</> Bash

ansible-playbook playbook.yml
```

Generated `/tmp/app.conf`:

```
</> Plain text

APP_NAME=payment-api
APP_PORT=8080
```

Understand:

- `.j2` is a conventional template-file extension.
- `template` renders the variables and writes the resulting file.
- The same template can generate different configurations for different environments.

Practical use: Generate application configuration files, Nginx configuration, service files, and environment-specific settings.

---

35. Conditions (`when`)

Meaning: Run a task only when a specified condition is true.

Example 1 — Check a variable:

```
</> YAML

- name: Condition example
  hosts: localhost
  connection: local

  vars:
    environment: production

  tasks:
    - name: Show production message
```

```
    ansible.builtin.debug:
      msg: "Production deployment"
    when: environment == "production"
```

Understand:

- If `environment` is `production`, the task runs.
- If it is `development`, the task is skipped.
- Do not put `{{ }}` around a variable in a `when` expression.

Example 2 — Check the operating system:

```
</> YAML

- name: Install package on Ubuntu
  ansible.builtin.apt:
    name: nginx
    state: present
  become: yes
  when: ansible_facts['distribution'] == "Ubuntu"
```

Example 3 — Use a registered result:

```
</> YAML

- name: Check a command
  ansible.builtin.command: systemctl is-active nginx
  register: nginx_status
  changed_when: false
  failed_when: false

- name: Show message if nginx is active
  ansible.builtin.debug:
    msg: "Nginx is active"
  when: nginx_status.stdout == "active"
```

Practical use: Install packages conditionally, skip unnecessary tasks, and perform different actions based on the server's state.

---

36. Loops

Meaning: Repeat the same task for multiple items.

Example: Install three packages without writing three separate tasks.

```yaml
- name: Loop example
  hosts: web
  become: yes

  tasks:
    - name: Install packages
      ansible.builtin.apt:
        name: "{{ item }}"
        state: present
      loop:
        - nginx
        - curl
        - git
```

Understand:

- `loop` contains the list of items.
- `item` represents the current item in each iteration.
- Ansible executes the task for each item.

Conceptually, Ansible performs:

```
Install nginx
Install curl
Install git
```

Example — Create multiple directories:

```yaml
- name: Create application directories
  ansible.builtin.file:
    path: "/opt/{{ item }}"
    state: directory
    mode: '0755'
  loop:
    - app
    - logs
    - backups
```

Practical use: Install multiple packages, create users, create directories, or manage several files with one task definition.

37. Loop Variables

Meaning: Values used by a loop can be simple strings or structured data such as dictionaries.

Example 1 — List of dictionaries

```yaml
- name: Create users
  hosts: localhost
  connection: local

  vars:
    users:
      - name: ram
        shell: /bin/bash
      - name: sam
        shell: /bin/sh

  tasks:
    - name: Display user details
      ansible.builtin.debug:
        msg: "User {{ item.name }} uses {{ item.shell }}"
      loop: "{{ users }}"
```

Output:

```
User ram uses /bin/bash
User sam uses /bin/sh
```

Understand:

- `item` is the current dictionary.
- `item.name` accesses the `name` field.
- `item.shell` accesses the `shell` field.

Example 2 — Custom loop variable

By default, Ansible uses `item`. You can give it a clearer name.

```yaml
- name: Display packages
  ansible.builtin.debug:
```

```
    msg: "Installing {{ package }}"
  loop:
    - nginx
    - curl
  loop_control:
    loop_var: package
```

Understand:

- `loop_var: package` changes the loop variable from `item` to `package`.
- Inside the task, use `{{ package }}`.

Practical use: Work with lists of users, applications, ports, or server configurations without confusing nested loop variables.

---

38. Filters

Meaning: Filters transform or process values using Jinja2's pipe (`|`) syntax.

Basic syntax:

```Jinja2
{{ variable | filter_name }}
```

Common filters

Filter	Purpose	Example
`default`	Supply a fallback value	`{{ app_port \
`upper`	Convert to uppercase	`{{ app_name \
`lower`	Convert to lowercase	`{{ app_name \
`int`	Convert to an integer	`{{ "8080" \
`string`	Convert to a string	`{{ 8080 \
`length`	Count items or characters	`{{ packages \
`join`	Combine list elements	`{{ packages \

Filter	Purpose	Example
replace	Replace text	`{{ "dev-server" \
to_json	Convert data to JSON text	`{{ config \
to_yaml	Convert data to YAML text	`{{ config \

Practical example

```yaml
- name: Filters example
  hosts: localhost
  connection: local

  vars:
    app_name: payment-api
    packages:
      - nginx
      - curl
      - git
    app_port: "8080"

  tasks:
    - name: Display transformed values
      ansible.builtin.debug:
        msg:
          - "Name: {{ app_name | upper }}"
          - "Package count: {{ packages | length }}"
          - "Packages: {{ packages | join(', ') }}"
          - "Port as integer: {{ app_port | int }}"
```

Expected output:

```yaml
- "Name: PAYMENT-API"
- "Package count: 3"
- "Packages: nginx, curl, git"
- "Port as integer: 8080"
```

Practical use: Format names, handle missing values, convert data types, and transform data before passing it to a task or template.

Important: The `default` filter normally substitutes a value when a variable is undefined. To also substitute for false-like values, use `default(8080, true)`.

Final revision — 13 concepts at a glance

Concept	Remember this
26. Variable definition	Store reusable values
27. Variable precedence	Decide which value wins
28. Host variables	Configure one server
29. Group variables	Share settings across servers
30. `-e`	Override values at runtime
31. Registered variables	Store task results
32. Facts	Discover host information
33. Magic variables	Access Ansible's built-in execution and inventory information
34. Jinja2	Insert variables into text and templates
35. `when`	Run tasks conditionally
36. Loops	Repeat tasks for multiple items
37. Loop variables	Access the current loop item
38. Filters	Transform values

Basic practical exercise

Try completing these five tasks using one playbook:

- [ ] Define `app_name` and `app_port` using `vars`.
- [ ] Print the host's name using `inventory_hostname`.

- [ ] Display the OS distribution using facts.
- [ ] Loop through `nginx`, `curl`, and `git` and print each package name.
- [ ] Convert `app_name` to uppercase using the `upper` filter.

Success criteria: You can explain what each concept does, identify where its value comes from, and write a basic example without referring to notes.